

LOG8415E

Advanced Concepts of Cloud Computing

Report

Jodel Alcindor
Department of Computer and Software Engineering
École Polytechnique de Montréal, Quebec, Canada
`marc-jodel-dumesle.alcindor@polymtl.ca`

1 Introduction

The objective of this project is to design and deploy a secure database access architecture using cloud design patterns. The system relies mainly on two patterns studied during the lectures: the Gatekeeper pattern and the Proxy pattern.

The architecture is deployed on AWS. The goal is to control how SQL queries are executed on a database cluster. Instead of allowing direct access to the database, all client requests must pass through a first service that is responsible for security checks and query routing.

The infrastructure is fully automated using Terraform. To deploy the services, we use a shell script. A benchmark is also used to evaluate the behavior and performance of different routing strategies.

2 Architecture Overview

The architecture is divided in four main components. First, we have a Gatekeeper service. It acts as a single entry point for all requests. Its role is to authenticate the requests and validate SQL queries before they reach the backend.

Second of all, we have a Proxy service that is responsible for routing validated queries to the appropriate database node. Then, we have the backend that consists of a MySQL cluster composed of one manager node and two worker nodes. Finally, we use a benchmark script to generate read and write workloads and measure performance.

With this design, we can ensure that the database cluster is never directly exposed to external clients.

3 Automation

The automation part plays an important role in this project. All cloud resources are provisioned using Terraform. We used and adapted all the modules developed in the previous laboratories. This includes the network module, the cluster module and the EC2 instances. Once the infrastructure is created, a deployment script installs dependencies and starts the Gatekeeper and Proxy services.

4 Implementation

4.1 Infrastructure Provisioning

The database cluster is composed of three EC2 instances. One instance acts as the manager node and is responsible for handling all write operations. The two remaining instances act as worker nodes and are mainly used to serve read queries.

All SQL queries executed during the benchmark are real queries executed against the MySQL cluster. Read operations use the Sakila dataset, while write operations insert rows into a dedicated benchmark table. This approach allows measuring the complete cost of request handling, including network latency, proxy routing logic, and database execution time.

4.2 Gatekeeper Service

The Gatekeeper service is implemented using FastAPI and runs on its own EC2 instance. It verifies that incoming requests contain a valid API key and applies basic SQL validation rules. To go further, we blocked potential dangerous queries to the database.

Only validated requests are forwarded to the Proxy service. This makes the Gatekeeper an effective security layer in front of the database cluster.

4.3 Proxy Service

The Proxy service is also implemented using FastAPI and is responsible for routing SQL queries. We implemented three strategies.

In the Direct strategy, all queries are sent to the manager node. In the Random strategy, read queries are distributed randomly across worker nodes, while write queries are always routed to the manager. In the Custom strategy, read queries are routed to the worker node with the lowest measured network latency. The custom strategy dynamically selects the worker with the lowest observed network latency.

5 Results

We used a benchmark to evaluate the behavior of the three routing strategies implemented in the proxy service. For each strategy, 1000 read queries and 1000 write queries were executed sequentially.

The measured execution times are summarized below:

- **Direct strategy:** 370.58 seconds for reads and 365.18 seconds for writes.

- **Random strategy:** 362.40 seconds for reads and 368.11 seconds for writes.
- **Custom strategy:** 709.76 seconds for reads and 2.70 seconds for writes.

With the Direct strategy, all queries are sent to the manager node. This creates a single bottleneck, which leads to relatively high but stable response times for both read and write operations.

The Random strategy attempts to improve performance by distributing read queries across worker nodes, while still routing all write queries to the manager to preserve consistency. However, the measured performance remains very close to the Direct strategy. This suggests that, for simple queries, network latency and proxy processing overhead have a greater impact than the choice of database node.

Then, we have the Custom strategy. It shows significantly worse performance for read operations. This can be explained by the cost of actively measuring network latency before routing each query.

Overall, the results highlight that naive load-balancing strategies and request latency measurements can degrade performance when not carefully optimized.

6 Conclusion

This project demonstrates how cloud design patterns can be applied to design a secure and modular database access architecture. The Gatekeeper and Proxy patterns effectively separate concerns between security enforcement, request routing, and data storage.

The experimental evaluation shows that while load-balancing strategies may appear beneficial in theory, their real-world effectiveness strongly depends on implementation details and system overhead.

Overall, this project highlights the importance of careful design and measurement when deploying cloud-based distributed systems.